

Filtering and Operators — Comparison, Logical, BETWEEN, IN, LIKE, Regex

Chapter 13 | PostgreSQL Complete Guide Module: 02_SQL_Basics | Difficulty: Beginner-Intermediate | Estimated Reading Time: 40 minutes

Table of Contents

- [Learning Objectives](#)
 - [Theory: Operators and Predicates](#)
 - [Comparison Operators](#)
 - [Logical Operators — AND, OR, NOT](#)
 - [BETWEEN](#)
 - [IN and NOT IN](#)
 - [LIKE and ILIKE](#)
 - [SIMILAR TO](#)
 - [Regular Expression Operators](#)
 - [ASCII Visual Diagrams](#)
 - [SQL Examples](#)
 - [Common Mistakes](#)
 - [Best Practices](#)
 - [Performance Considerations](#)
 - [Interview Questions](#)
 - [Interview Answers](#)
 - [Hands-on Exercises](#)
 - [Solutions](#)
 - [Advanced Notes](#)
 - [Cross-References](#)
-

Learning Objectives

By the end of this chapter, you will be able to:

- Apply all PostgreSQL comparison operators correctly, including `IS DISTINCT FROM`
 - Combine conditions with AND, OR, and NOT using correct operator precedence
 - Use BETWEEN, IN, LIKE, and ILIKE for range, list, and pattern matching
 - Write SIMILAR TO patterns and understand when they differ from LIKE
 - Use PostgreSQL-specific regex operators `~`, `~*`, `!~`, `!~*` for full regular expression matching
-

Theory: Operators and Predicates

An **operator** takes one or two operands and returns a value. In WHERE clauses the return value must be Boolean (TRUE/FALSE/NULL). A **predicate** is an operator or keyword that specifically returns a Boolean, used to filter rows.

Operator Categories

Category	Operators
----------	-----------

Comparison	=, <>, !=, <, >, <=, >=
Identity	IS NULL, IS NOT NULL, IS DISTINCT FROM, IS NOT DISTINCT FROM
Range	BETWEEN ... AND ..., NOT BETWEEN
Membership	IN (...), NOT IN (...)
Pattern matching	LIKE, NOT LIKE, ILIKE, NOT ILIKE
Regex (SQL standard)	SIMILAR TO, NOT SIMILAR TO
Regex (PostgreSQL)	~, ~*, !~, !~*
Logical	AND, OR, NOT

Comparison Operators

Standard Comparisons

```
-- Equal
SELECT * FROM employees WHERE department_id = 10;

-- Not equal (two syntaxes, identical behavior)
SELECT * FROM employees WHERE department_id <> 10;
SELECT * FROM employees WHERE department_id != 10;

-- Less than / greater than
SELECT * FROM employees WHERE salary < 50000;
SELECT * FROM employees WHERE salary > 100000;

-- Less than or equal / greater than or equal
SELECT * FROM employees WHERE hire_date <= '2020-12-31';
SELECT * FROM employees WHERE hire_date >= '2015-01-01';
```

IS DISTINCT FROM and IS NOT DISTINCT FROM

Standard `=` returns NULL when either operand is NULL. `IS DISTINCT FROM` treats NULLs as equal to each other (NULL IS DISTINCT FROM NULL → FALSE).

```
-- Standard = with NULL: always returns NULL (never TRUE)
SELECT NULL = NULL;           -- returns NULL, not TRUE
SELECT NULL = 'something';    -- returns NULL

-- IS DISTINCT FROM: NULL-safe comparison
SELECT NULL IS DISTINCT FROM NULL;      -- FALSE (they are the same)
SELECT NULL IS DISTINCT FROM 'something'; -- TRUE (they are different)
SELECT 1 IS DISTINCT FROM 2;             -- TRUE
SELECT 1 IS NOT DISTINCT FROM 1;         -- TRUE

-- Practical use: compare nullable columns
```

```
SELECT * FROM employees
WHERE manager_id IS NOT DISTINCT FROM NULL; -- equivalent to IS NULL, but
composable
```

Logical Operators — AND, OR, NOT

Operator Precedence

NOT binds tighter than AND, which binds tighter than OR. Use parentheses to be explicit.

Precedence (high to low):

1. NOT
2. AND
3. OR

```
-- This reads: A AND (B OR C) because AND binds tighter than OR
-- To change precedence, use parentheses:
SELECT * FROM orders
WHERE status = 'pending'
  AND (total > 1000 OR priority = 'high');
```

Truth Tables

AND truth table:	OR truth table:	NOT truth table:
T AND T = T	T OR T = T	NOT T = F
T AND F = F	T OR F = T	NOT F = T
T AND NULL = NULL	T OR NULL = T	NOT NULL = NULL
F AND F = F	F OR F = F	
F AND NULL = F	F OR NULL = NULL	
NULL AND NULL = NULL	NULL OR NULL = NULL	

Note: `TRUE OR NULL = TRUE` (short-circuit), but `FALSE AND NULL = FALSE` (short-circuit).

BETWEEN

BETWEEN is inclusive on both ends. `x BETWEEN a AND b` is exactly `x >= a AND x <= b`.

```
-- Numeric range
SELECT * FROM products WHERE unit_price BETWEEN 10 AND 50;

-- Date range
SELECT * FROM orders
WHERE order_date BETWEEN '2023-01-01' AND '2023-12-31';

-- NOT BETWEEN
SELECT * FROM products WHERE unit_price NOT BETWEEN 10 AND 50;
```

BETWEEN with timestamps

```
-- Include all of 2023-06-15 (use < next day for timestamp columns)
-- BETWEEN with TIMESTAMP is inclusive of the endpoint second
SELECT * FROM events
WHERE event_time BETWEEN '2023-06-15 00:00:00' AND '2023-06-15 23:59:59';

-- Safer pattern (avoids sub-second edge cases):
SELECT * FROM events
WHERE event_time >= '2023-06-15'
AND event_time < '2023-06-16';
```

IN and NOT IN

IN tests membership in a list or subquery result.

IN with a Value List

```
-- List of literals
SELECT * FROM employees WHERE department_id IN (10, 20, 30);

-- Equivalent verbose form:
SELECT * FROM employees
WHERE department_id = 10
OR department_id = 20
OR department_id = 30;
```

NOT IN — Critical NULL Trap

```
-- NOT IN with a list containing NULL returns UNKNOWN for all rows
-- This query returns ZERO rows if any department_id in the list is NULL!
SELECT * FROM employees WHERE department_id NOT IN (10, NULL, 30);
-- Because: 5 <> NULL evaluates to NULL, so NOT IN short-circuits to UNKNOWN
```

IN with Subquery

```
SELECT * FROM employees
WHERE department_id IN (
    SELECT department_id FROM departments WHERE location_id = 1700
);
```

NOT IN with Subquery — Safer Alternative

```
-- If the subquery can return NULLs, NOT IN is broken. Use NOT EXISTS instead:
SELECT * FROM employees e
WHERE NOT EXISTS (
    SELECT 1 FROM departments d
    WHERE d.department_id = e.department_id
```

```
        AND d.location_id = 1700
    );
```

LIKE and ILIKE

LIKE Wildcards

Wildcard	Meaning
%	Any sequence of zero or more characters
_	Any single character

```
-- Starts with 'Jo'
SELECT * FROM employees WHERE last_name LIKE 'Jo%';

-- Ends with 'son'
SELECT * FROM employees WHERE last_name LIKE '%son';

-- Contains 'mit'
SELECT * FROM employees WHERE last_name LIKE '%mit%';

-- Exactly 4 characters
SELECT * FROM employees WHERE last_name LIKE '____';

-- Second character is 'a'
SELECT * FROM employees WHERE last_name LIKE '_a%';
```

ILIKE — Case-Insensitive (PostgreSQL Extension)

```
-- Case-insensitive LIKE
SELECT * FROM customers WHERE email ILIKE '%@gmail.com';
SELECT * FROM products WHERE product_name ILIKE 'widget%';
```

Escaping Wildcards in LIKE

```
-- Use ESCAPE to match literal % or _
SELECT * FROM notes WHERE content LIKE '100\%' ESCAPE '\';
SELECT * FROM files WHERE filename LIKE 'file\_backup' ESCAPE '\';
```

NOT LIKE / NOT ILIKE

```
SELECT * FROM products WHERE product_name NOT LIKE 'Widget%';
SELECT * FROM customers WHERE email NOT ILIKE '%@test.com';
```

SIMILAR TO

SIMILAR TO blends SQL LIKE syntax with basic regex. It is more powerful than LIKE but less powerful than full regex. The pattern is anchored at both ends (implicit `^` and `$`).

SIMILAR TO Metacharacters

Symbol	Meaning
<code>%</code>	Any sequence (like LIKE's <code>%</code>)
<code>_</code>	Any single character (like LIKE's <code>_</code>)
<code>\</code>	<code>\</code>
<code>*</code>	Zero or more of previous
<code>+</code>	One or more of previous
<code>?</code>	Zero or one of previous
<code>{n}</code>	Exactly n repetitions
<code>[abc]</code>	Character class
<code>(...)</code>	Grouping

```
-- Phone number format check: optional country code then digits
SELECT * FROM contacts
WHERE phone SIMILAR TO '(\+1-)?[0-9]{3}-[0-9]{3}-[0-9]{4}';

-- Either 'cat' or 'dog'
SELECT * FROM pets WHERE species SIMILAR TO 'cat|dog';

-- String of 5-8 alphanumeric characters
SELECT * FROM users WHERE username SIMILAR TO '[A-Za-z0-9]{5,8}';
```

Regular Expression Operators

PostgreSQL provides POSIX regular expression operators. These are more powerful and usually faster than SIMILAR TO.

Operator	Meaning
<code>~</code>	Matches regex (case-sensitive)
<code>~*</code>	Matches regex (case-insensitive)
<code>!~</code>	Does NOT match regex (case-sensitive)
<code>!~*</code>	Does NOT match regex (case-insensitive)

```
-- Emails containing a digit in the local part
SELECT * FROM users WHERE email ~ '^[^@]*[0-9][^@]*@';

-- Case-insensitive match for 'admin' anywhere in username
SELECT * FROM users WHERE username ~* 'admin';

-- NOT matching a pattern
SELECT * FROM products WHERE sku !~ '^[A-Z]{2}[0-9]{4}$';

-- Validate format with case-insensitive regex
SELECT * FROM employees WHERE phone ~* '^\+?[0-9\s\-\(\)]{7,20}$';
```

Regex Functions

```
-- Extract first match
SELECT regexp_match('2024-06-15', '(\d{4})-(\d{2})-(\d{2})');
-- Returns: {'2024','06','15'}

-- All matches
SELECT regexp_matches('cat bat rat', '[a-z]at', 'g');

-- Replace with regex
SELECT regexp_replace('Hello World', '\s+', ' ', 'g');
-- Returns: 'Hello World'
```

ASCII Visual Diagrams

Operator Precedence Tree

Expression: A OR B AND NOT C

Parsed as: A OR (B AND (NOT C))

```

      OR
     /  \
    A    AND
       /  \
      B    NOT
       |
      C
```

BETWEEN Range (Inclusive)

```
number line:
... 8   9   [10  11  12  13  14  15]  16  17 ...
      ^               ^
    BETWEEN 10       AND 15
```

```
WHERE price BETWEEN 10 AND 15  includes 10 and 15 themselves
```

LIKE Pattern Matching

Pattern: 'J%n'

Matches:

```
"John"      -> J [oh] n  YES
"Jan"        -> J [] n    YES (% matches zero chars)
"Jonathan"   -> J [onatha] n  YES
"Jane"       -> J [an] e  NO  (doesn't end in 'n')
```

Pattern: 'J__n'

```
"John"      -> J [oh] n  YES (exactly 2 chars between J and n)
"Jan"        -> J [a] n   NO  (only 1 char, need exactly 2)
"Joon"       -> J [oo] n  YES
```

NOT IN with NULL — The Trap

Query: department_id NOT IN (10, NULL, 30)

Evaluation for row where department_id = 5:

```
5 <> 10    -> TRUE
5 <> NULL   -> NULL  <-- short-circuit! NOT IN sees UNKNOWN
Result: UNKNOWN (not TRUE), row is excluded
```

Result: ZERO rows returned (all rows are excluded due to NULL in list)

SQL Examples

```
-- Example 1: Equality and inequality
SELECT * FROM products WHERE category = 'Electronics';
SELECT * FROM products WHERE category <> 'Electronics';

-- Example 2: Greater than / less than
SELECT * FROM orders WHERE total_amount > 500;
SELECT * FROM employees WHERE hire_date < '2018-01-01';

-- Example 3: IS DISTINCT FROM for NULL-safe comparison
SELECT * FROM employees
WHERE manager_id IS NOT DISTINCT FROM NULL;

-- Example 4: AND with parentheses
SELECT * FROM orders
WHERE status = 'shipped'
      AND (total_amount > 1000 OR express_delivery = TRUE);
```

```

-- Example 5: OR conditions
SELECT * FROM products
WHERE category = 'Books'
      OR category = 'Music';

-- Example 6: NOT operator
SELECT * FROM employees
WHERE NOT (department_id = 10 OR department_id = 20);

-- Example 7: BETWEEN numeric
SELECT * FROM products WHERE unit_price BETWEEN 5.00 AND 25.00;

-- Example 8: BETWEEN date
SELECT order_id, order_date, total_amount
FROM orders
WHERE order_date BETWEEN '2023-01-01' AND '2023-06-30';

-- Example 9: NOT BETWEEN
SELECT * FROM employees WHERE salary NOT BETWEEN 40000 AND 80000;

-- Example 10: IN with list
SELECT * FROM departments WHERE department_name IN ('Sales', 'IT', 'HR', 'Finance');

-- Example 11: IN with subquery
SELECT first_name, last_name
FROM employees
WHERE job_id IN (SELECT job_id FROM jobs WHERE min_salary >= 10000);

-- Example 12: NOT IN (safe – no NULLs in subquery)
SELECT * FROM products
WHERE product_id NOT IN (SELECT product_id FROM order_details);

-- Example 13: LIKE starts-with
SELECT * FROM customers WHERE last_name LIKE 'Mac%';

-- Example 14: LIKE single-char wildcard
SELECT * FROM customers WHERE last_name LIKE '_rown';

-- Example 15: ILIKE case-insensitive
SELECT * FROM products WHERE description ILIKE '%wireless%';

-- Example 16: NOT ILIKE
SELECT * FROM users WHERE email NOT ILIKE '%@disposable.%';

-- Example 17: SIMILAR TO alternation
SELECT * FROM contacts WHERE phone SIMILAR TO '555-[0-9]{3}-[0-9]{4}|800-[0-9]{3}-[0-9]{4}';

-- Example 18: Regex ~ case-sensitive
SELECT * FROM employees WHERE last_name ~ '^[A-M]'; -- last name starts A-M

-- Example 19: Regex ~* case-insensitive

```

```
SELECT * FROM products WHERE description ~* 'bluetooth|wireless|wifi';

-- Example 20: Regex !~ does NOT match
SELECT * FROM users WHERE username !~ '^[a-zA-Z0-9_]'; -- only alphanumeric +
underscore
```

Common Mistakes

1. **NOT IN with NULL in the list.** If the list passed to NOT IN contains a NULL (or if a subquery returns any NULL), NOT IN returns UNKNOWN for every row. Use NOT EXISTS or filter NULLs from the subquery: WHERE col NOT IN (SELECT other_col FROM t WHERE other_col IS NOT NULL) .
2. **Forgetting BETWEEN is inclusive.** BETWEEN 1 AND 10 includes both 1 and 10. If you want to exclude an endpoint, use >= 1 AND < 10 instead.
3. **LIKE is case-sensitive by default.** In PostgreSQL, LIKE 'hello%' will NOT match 'Hello'. Use ILIKE for case-insensitive search.
4. **SIMILAR TO is anchored.** Unlike regex ~, SIMILAR TO implicitly anchors the pattern to the full string (adds ^...\$). SIMILAR TO 'cat' only matches the exact string "cat", not a string containing "cat".
5. **Operator precedence surprises.** WHERE a = 1 OR b = 2 AND c = 3 is evaluated as a = 1 OR (b = 2 AND c = 3) . Always use parentheses to make intent explicit.
6. **Using = to compare with NULL.** WHERE col = NULL always returns UNKNOWN (never TRUE). Always use IS NULL or IS NOT NULL .
7. **Large IN lists causing plan issues.** An IN list with thousands of values may cause the planner to choose a suboptimal plan. Consider a temporary table or = ANY(ARRAY[. . .]) for very large sets.

Best Practices

1. **Use IS NULL / IS NOT NULL** , never = NULL or <> NULL .
2. **Use NOT EXISTS instead of NOT IN when the subquery could return NULLs** — it is both safer and often faster.
3. **Parenthesize complex Boolean expressions** even when not strictly necessary — makes intent clear and prevents precedence bugs during code review.
4. **Prefer regex ~ over SIMILAR TO** — SIMILAR TO is less well-known, less powerful, and often slower. Use LIKE for simple patterns, ~ for complex ones.
5. **Anchor regex patterns when needed.** ~ '^prefix' matches at the start; ~ 'suffix\$' matches at the end; ~ 'pattern' without anchors matches anywhere in the string.
6. **For list membership, prefer = ANY(ARRAY[. . .]) for parameterized queries** — it works cleanly with prepared statements and avoids generating a new plan per list size.

7. Avoid functions on indexed columns in WHERE. WHERE LOWER(email) = 'x' prevents index use; instead create a functional index or use ILIKE .

Performance Considerations

Index Use by Operator

Condition	B-tree Index Used?	Notes
col = value	Yes	Most efficient
col > value	Yes (range scan)	Uses index
col BETWEEN a AND b	Yes (range scan)	Equivalent to >= AND <=
col IN (list)	Yes (multiple scans)	Planner may use bitmap OR
col LIKE 'prefix%'	Yes (if prefix)	Only when pattern has leading literal
col LIKE '%suffix'	No	Leading wildcard prevents index
col ILIKE 'prefix%'	Only with citext or functional index	Requires special setup
col ~ '^prefix'	Only with trigram index (pg_trgm)	Or pg_regex index
col ~ 'pattern'	Only with trigram index	pg_trgm extension

Trigram Indexes for LIKE/ILIKE/regex

```
-- Enable pg_trgm for index-accelerated pattern matching
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE INDEX idx_products_name_trgm
  ON products USING gin(product_name gin_trgm_ops);

-- Now these all use the index:
SELECT * FROM products WHERE product_name ILIKE '%wireless%';
SELECT * FROM products WHERE product_name ~ 'wire';
```

IN vs = ANY(ARRAY[...])

For dynamic queries and prepared statements, = ANY(\$1) can be more efficient because the plan is not re-generated for each different list size:

```
SELECT * FROM employees WHERE department_id = ANY($1::int[]);
```

Interview Questions

1. What is the difference between = and IS NOT DISTINCT FROM ?
2. Why can NOT IN return zero rows even when data seems to match?

3. What is the difference between LIKE and ILIKE in PostgreSQL?
 4. How does SIMILAR TO differ from both LIKE and the ~ operator?
 5. When should you use NOT EXISTS instead of NOT IN ?
 6. What is operator precedence for AND and OR, and how do you override it?
 7. Can a LIKE pattern use an index? Under what conditions?
 8. What is the ~* operator and when would you use it?
 9. How would you efficiently search for a substring anywhere in a large text column?
 10. Explain the three-valued logic of SQL Boolean expressions.
-

Interview Answers

Q1: = vs IS NOT DISTINCT FROM ? = returns NULL if either operand is NULL (NULL comparison always yields UNKNOWN). IS NOT DISTINCT FROM is NULL-safe: NULL IS NOT DISTINCT FROM NULL returns TRUE. Use IS NOT DISTINCT FROM when comparing nullable columns for equality.

Q2: NOT IN returns zero rows? NOT IN (list) evaluates as multiple <> checks combined with AND. If any value in the list is NULL, then col <> NULL is UNKNOWN, and the whole NOT IN expression becomes UNKNOWN (not TRUE). Since WHERE requires TRUE, all rows are excluded. Solution: use NOT EXISTS, or ensure the subquery filters out NULLs.

Q3: LIKE vs ILIKE? LIKE is case-sensitive ('hello%' does not match 'Hello'). ILIKE is a PostgreSQL extension that performs case-insensitive matching. There is no ILIKE in standard SQL — the equivalent is LOWER(col) LIKE LOWER(pattern) .

Q4: SIMILAR TO vs LIKE vs ~ ? LIKE supports only % and _ wildcards. SIMILAR TO adds regex-like syntax (| , * , + , []) but anchors the pattern to the full string. ~ uses full POSIX regex without anchoring — matches anywhere in the string unless ^ / \$ are used. ~ is the most powerful and generally preferred for complex patterns.

Q5: NOT EXISTS vs NOT IN? NOT EXISTS is NULL-safe and stops searching as soon as a match is found (efficient). NOT IN computes the entire subquery list and breaks if any NULL is present. For correlated subqueries checking non-membership, NOT EXISTS is always safer and often faster.

Q6: AND/OR precedence? NOT > AND > OR. A OR B AND C = A OR (B AND C) . Override with explicit parentheses.

Q7: LIKE and indexes? LIKE can use a B-tree index only when the pattern has a fixed prefix with no leading wildcard: col LIKE 'abc%' uses the index; col LIKE '%abc' does not. For any-position substring matching, create a GIN trigram index with pg_trgm.

Q8: ~* operator? ~* performs a case-insensitive POSIX regex match. It is the case-insensitive version of ~ . Use it when you need regex power plus case insensitivity, such as searching for user-provided terms in text fields.

Q9: Efficient substring search? Enable pg_trgm and create a GIN index with gin_trgm_ops . This accelerates both LIKE '%term%' and ~ 'term' queries by decomposing strings into trigrams and using inverted indexes.

Q10: Three-valued logic? SQL Boolean expressions evaluate to TRUE, FALSE, or UNKNOWN (NULL). WHERE only passes rows where the condition is TRUE; FALSE and UNKNOWN (NULL) both exclude the row. This is why NOT (col = NULL) does not return rows where col is NULL — col = NULL is UNKNOWN, and NOT UNKNOWN is still UNKNOWN.

Hands-on Exercises

Setup: Use the products table from `03_select_basics.md`.

Exercise 1: Find all products that are either in 'Electronics' or 'Home' categories, have a unit price between \$10 and \$100, and are not discontinued.

Exercise 2: Find all products whose name contains the letter 'a' (case-insensitive) using ILIKE.

Exercise 3: Find all products whose name matches the regex pattern: starts with a capital letter, followed by at least one word character, then a space, then another capital letter. Use the `~` operator.

Exercise 4: Find products that are NOT in the 'Electronics' or 'Industrial' categories using NOT IN.

Exercise 5: Write a query that demonstrates the NOT IN NULL trap: create a scenario where NOT IN returns 0 rows because of a NULL in the exclusion list, then fix it with NOT EXISTS.

Solutions

```
-- Exercise 1
SELECT product_name, category, unit_price
FROM products
WHERE category IN ('Electronics', 'Home')
      AND unit_price BETWEEN 10 AND 100
      AND discontinued = FALSE;

-- Exercise 2
SELECT product_name FROM products WHERE product_name ILIKE '%a%';

-- Exercise 3
SELECT product_name FROM products WHERE product_name ~ '^[A-Z]\w+ [A-Z]';

-- Exercise 4
SELECT product_name, category
FROM products
WHERE category NOT IN ('Electronics', 'Industrial');

-- Exercise 5: NOT IN trap
-- Insert a product with NULL category
INSERT INTO products (product_name, unit_price, units_in_stock)
VALUES ('Mystery Item', 9.99, 10); -- category is NULL

-- This returns 0 rows because NULL is in the IN list via the subquery
-- (simulated by adding NULL directly)
SELECT * FROM products WHERE category NOT IN ('Electronics', NULL);

-- Fix with NOT EXISTS:
SELECT p.* FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM (VALUES ('Electronics'), (NULL)) AS excluded(cat)
    WHERE excluded.cat = p.category
```

```
);  
-- Or more practically:  
SELECT * FROM products WHERE category IS DISTINCT FROM 'Electronics';
```

Advanced Notes

The ANY and ALL Quantifiers

`= ANY(array)` is equivalent to `IN`. `<> ALL(array)` is equivalent to `NOT IN` but is NULL-safe when using arrays:

```
SELECT * FROM employees WHERE department_id = ANY(ARRAY[10, 20, 30]);  
SELECT * FROM employees WHERE department_id <> ALL(ARRAY[10, 20, 30]);
```

Row Constructors in Comparisons

PostgreSQL supports comparing row values:

```
SELECT * FROM shipments  
WHERE (warehouse_id, product_id) IN (SELECT warehouse_id, product_id FROM  
low_stock);
```

Operator Classes and Custom Operators

PostgreSQL allows defining custom operators. You can even create operators that behave like `<` but for custom data types (e.g., IP ranges, geometric types). This is relevant when working with PostGIS or hstore extensions.

`~~` and `~~*` Operators

`LIKE` is syntactic sugar for the `~~` operator; `ILIKE` is syntactic sugar for `~~*`. You may see these in `EXPLAIN` output or `pg_operator` catalog:

```
SELECT 'hello' ~~ 'hel%'; -- TRUE (same as LIKE)  
SELECT 'Hello' ~~* 'hel%'; -- TRUE (same as ILIKE)
```

Cross-References

- **Previous:** [03_select_basics.md](#) — `SELECT`, `WHERE`, `ORDER BY`, `LIMIT`
- **Next:** [05_null_handling.md](#) — `NULL` semantics, `IS NULL`, `COALESCE`, `NULLIF`
- **Related:** [06_string_functions.md](#) — `REGEXP_REPLACE`, `REGEXP_MATCHES`
- **Related (03_Intermediate):** [03_Intermediate_SQL/05_subqueries.md](#) — `IN/NOT IN` with subqueries, `EXISTS`
- **Related (03_Intermediate):** [03_Intermediate_SQL/06_indexes.md](#) — Creating indexes to support filter queries